CassaCheck: Technical Report

BSE, African Leadership University.
Supervisor: Emmanuel Adjei.
Repository: https://github.com/Justine-abc/cassavacheck
Walkthrough video: https://youtu.be/vphYf2nHZVs?si=g2AHyz8kFxENgMlb
Date: 26 October 2026.

CassaCheck classifies a photograph of a cassava leaf into one of five conditions (cassava bacterial blight, brown streak disease, green mite damage, mosaic disease, healthy), reports a calibrated confidence, abstains when that confidence is below a threshold fixed on validation data, and shows published district-level disease information beside the result without combining the two. This report covers scope alignment, technical functionality, the algorithms, testing and performance evidence, and the code-quality practices of the submitted version.

1. Scope alignment and feature validation

The approved proposal (6 October 2026) defines six functional requirements, six non-functional requirements, a data protocol and an evaluation plan. The table maps each planned feature to its status in this version.

Planned feature	Intended purpose	Current status	Evidence
FR-01 Accept a leaf photo from a mobile browser; validate type and size	Input without location or personal data	Completed	api/main.py /diagnose: 415 on wrong type, 413 over 10 MB, 422 on a corrupt file; tests/test_api.py
FR-02 District and season from lists; published incidence shown separately with survey year and denominator	Site information beside the result, never inside the model	Completed for the two districts verified from the 2019 survey; the other 11 and the CMD column to be added from the same paper	api/districts.py; data/district_prevalence.csv; panel screenshots
FR-03 Calibrated five class probabilities	Honest confidence	Completed: temperature 0.931 fitted on a validation half	api/calibration.py Calibrator; models/calibration.json; notebook 04
FR-04 Frozen abstention threshold; label or abstention	Say not sure instead of guessing	Completed: threshold 0.6455 for target coverage 0.80; realized coverage	AbstentionGate; risk-coverage figure; notsure screenshot

		0.79	
FR-05 Result card with advice, or three follow-up questions and referral	Management advice, never a pesticide	Completed	api/advice.py; api/static/index.html ; forbidden-word test
FR-06 Anonymised request log	Evaluation and officer view	Completed with SQLite; PostgreSQL on Render planned	api/log_store.py; GET /log
Near-duplicate detection and published 70/15/15 split	Prevent leakage	Completed: 0 fingerprint collisions at hash size 8; every class in every split	notebook 01; data/split_v1_seed42.csv
MobileNetV3-Large and ResNet-18 baselines, five seeds	Backbone comparison with uncertainty	Partial: MobileNetV3-Large, one seed, two epochs (validation accuracy 0.8237, macroF1 0.7109); ResNet-18 implemented, not trained; five seeds scheduled by 2 November	api/classifier.py; notebook 02
External evaluation on CCMT, once, with class mapping	Cross-country transfer	Not yet; protocol fixed (proposal Table 5); scheduled by 9 November	proposal 3.2.4
React interface from the Stitch designs	Mobile-first interface	Partial: six screens designed and exported; current interface is one static page with the same flow	docs/design/*.html
Deployment on Render and Vercel	Public demonstration	Not yet; runs locally with one command	README

Justification of deviations: this course deadline (23 October) falls before phases 2 to 4 of the proposal's schedule. The calibrated, abstaining five-class classifier, the interface and the log are complete; the multi-seed comparison, the external evaluation and the deployment remain scheduled and are not claimed.

2. Technical functionality

Backend and frontend interaction

The page posts one multipart form (image, district, season) to POST /diagnose. The API returns a single JSON object: decision (a class or abstain), calibrated confidence, the frozen threshold, the five probabilities, advice lines, follow-up questions when abstaining, a referral flag, the district sentence and the model version. The page renders the confident state or the not-sure state from that one response; GET /districts fills the district list; GET /log serves the extension-officer view; GET /health reports whether weights are loaded and which calibration values are active. Swagger UI is generated at /docs.

Data handling

The image is checked by content type (JPEG or PNG), size (10 MB) and decodability, resized to 224 by 224 and normalised; it is never written to disk. The district table is loaded once from CSV into frozen records. One log row per request holds district, season, decision, confidence, timestamp and model version; no image bytes, coordinates or names.

Logic

Logits from the classifier; division by the temperature and softmax (calibration); comparison of the top probability with the threshold (abstention); advice or questions selected from rule data; district lookup with season fallback. District information never enters the probabilities: the panel is computed from the user's choice alone, and the tests assert that a diagnosis response is unchanged by the district field.

3. Algorithms and technical problem solving

● Near-duplicate detection and stratified group split (notebook 01). A perceptual hash gives near-identical photographs the same 64-bit fingerprint; StratifiedGroupKFold then splits 21,397 images 70/15/15 by class with fingerprints as groups, so a copy can never sit on both sides of a boundary. Result: 0 collisions at hash size 8; train 14,977, validation 3,210, test 3,210; the split is published with image identifiers.

● Class-weighted cross-entropy (notebook 02). CMD is 61% of labels; weights inverse to class frequency (CBB 19.7 against CMD 1.6) raised CBB recall from 0.41 to 0.64 and CBSD recall from 0.49 to 0.77 compared with the unweighted run, at the cost of some healthy-class precision.

● Temperature scaling by L-BFGS (notebook 04). One scalar divides the logits; it is found by minimising negative log-likelihood on a stratified half of the validation split (1,605 images) and measured on the other half. Result: T = 0.931; expected calibration error 0.0275 before and 0.0296 after; negative loglikelihood 0.5066 to 0.5040. Interpretation: the two-epoch model is not over-confident, so calibration is neutral at this stage; it stays in the pipeline because over-confidence grows with longer training (Guo et al., 2017), which the five-seed runs will test.

● Abstention threshold as a quantile (notebook 04). The threshold is the 20th percentile of calibrated topprobability on the fit half, so the model answers the most confident 80%. On the other half: realized coverage 0.791; accuracy 0.907 on retained answers against 0.825 on all; accuracy 0.513 on the abstained 21%, which shows the gate selects the hard cases rather than random ones.

● Expected calibration error, reliability diagram and risk-coverage curve implemented from their definitions (15 bins; coverage swept over confidence rank), as given in the proposal's Section 3.2.4.

● Engineering problem solved during testing: direct camera capture from the page is blocked by mobile browsers over plain HTTP (secure-context rule). The input was changed to a general picker, which offers camera and files on all tested browsers; direct capture returns under HTTPS on deployment.

4. Testing approaches and results

Testing approaches and results

Why: calibration, the gate, settings loading, district lookup and the advice rules are the decision logic, and must be correct independent of any model or image. How: tests/test_calibration.py and tests/test_districts_and_advice.py construct logits and tables directly. Results: all pass. Improvement made: the gate originally accepted a threshold above 1; a validation and a test were added; the advice test fails if any line contains a pesticide, fungicide, insecticide, spray or dose.

Strategy 2: integration testing through HTTP (pytest with FastAPI TestClient, 5 tests)

Why: the page depends on the response shape. How: health, districts, a text file (expects 415), a corrupt JPEG (expects 422), and a full diagnosis with the real weights checking the five keys, probabilities summing to one, and the rule that decision is abstain exactly when confidence is below the threshold. Results: 15 of 15 pass in 3.3 s (screenshot in docs/screenshots).

Strategy 3: functional testing with different inputs

Input	Expected behaviour	Observed
Field photo of a leaf, Kigali, clear	Label with confidence above 65%	Cassava mosaic disease, 76%; green mite, 74% on a second photo

Field photo, ambiguous leaf	Abstention with three questions	Not sure: green mite 59%, brown streak 34%; questions and referral shown
Text file	415 with a message	415 Upload a JPEG or PNG photograph
Corrupt JPEG bytes	422	422 The file is not a readable image
District with no survey row for the chosen season	Fallback to the district's row	Bugesera season C shows the season A row
Unknown district	No panel, no invented value	Panel hidden

Strategy 4: performance

Forward pass, batch size 1, one CPU thread, median of 100 runs after warm-up (scripts/measure_latency.py), on an HP ProBook 430 G8 (Intel Core i7-1165G7, 8 GB RAM), Ubuntu 26.04, PyTorch 2.3.1 CPU: MobileNetV3- Large 19.8 ms median, 21.8 ms at the 95th percentile; ResNet-18 architecture 61.7 ms, which justifies the MobileNetV3 choice. End-to-end /diagnose in Chrome on the laptop: 261 ms. End-to-end from the phone over a hotspot connection: under 1 s, below stopwatch resolution. Target: forward pass under 100 ms, request under 3 s on 3G; both met with margin.

Compatibility

Chrome and Firefox on Ubuntu, Safari on an iPhone 14 Pro at phone width: identical card, bars, questions and panel. One defect found and fixed (camera capture over HTTP, Section 3). Not yet tested: Android Chrome and low-end devices; the latency margin suggests the forward pass is not the bottleneck there, the image upload is.

5. Code quality practices

Separation of concerns: the API is seven modules with one responsibility each, and they are the classes of the proposal's class diagram (Figure 9): ImageClassifier, Calibrator, AbstentionGate, Decision, DistrictRepository, AdvisoryService, DiagnosisLog; api/main.py contains routes only. Abstraction and inheritance: ImageClassifier is an abstract base that fixes the contract (an image in, five logits out); MobileNetV3Classifier and ResNet18Classifier implement build_network, and the API depends only on the base, so a backbone is swapped in one line. Encapsulation: Decision, CalibrationSettings and DistrictPrevalence are frozen dataclasses; tuned values live in models/calibration.json, never in code, and a missing file falls back to an uncalibrated, neverabstaining configuration that announces itself. Polymorphism: any classifier subclass is interchangeable at startup. Rules as data: advice and follow-up questions are data structures edited without a deploy, guarded by a

test. Naming and documentation: full words, long-form command-line flags, comments that explain why, a CONTRIBUTING file stating the design constraints. Repository: notebooks for research, api for the product, tests, scripts, docs with designs and screenshots, pinned requirements.

6. Limitations and next steps

One seed and two epochs; calibration neutral at this stage; two districts in the table; ResNet-18 untrained; external evaluation and deployment scheduled; the designed interface not yet built in React. Next: five-seed runs of both backbones (by 2 November), the single external evaluation on CCMT with the class mapping of the proposal (by 9 November), completion of the district table, Render deployment with HTTPS, and the React interface from the exported designs.